Final Report: Song Recommendation Project

Link for our project on Git Hub: https://github.com/simongydvandy/Song-Rec-Project-for-CS-3262 It has all the data and codes we used for the project as well as this Readme final analysis report.

Team: Simon (Yiding) Gou (gouy) - Xizhi Li (lix71)

1. Problem Statement

The recommendation task involves developing a music recommendation system using a 172-entry survey dataset containing Vanderbilt students' demographic information and song preferences. The approach compares distinct machine learning methodologies to generate recommendations. A K-Nearest Neighbors (KNN) model is utilized for similarity-based recommendations, combining content-based and collaborative filtering. An unsupervised K-means clustering model was initially implemented but yielded poor recommendations. A supervised Random Forest model was subsequently developed to replace the clustering approach and improve predictive accuracy.

Overview

Approach	Owner	Type	
EDA	Simon and Xizhi		
KNN (content-based + collaborative filtering)	Simon	Similarity-based	
K-means clustering	Xizhi	Unsupervised clustering	(This was done first but has a poor recommendation, so the Random Forest is done later)
Randrom Forest	Xizhi	Supervised learning	

Note: For EDA and data preprocessings, Simon and Xizhi finished them collaboratively.

Project Structure

```
├── Song Recommendations.csv   # Raw survey data
├── src/
│   ├── preprocess.py          # Cleaning + feature engineering (shared)
│   ├── content_knn.py         # Content-based KNN (Simon)
│   ├── collab_knn.py          # User-based collaborative KNN (Simon)
│   ├── kmeans_recommender.py  # K-means clustering recommender (Xizhi)
│   └── evaluate.py            # Shared evaluation metrics
└── notebooks/
    ├── 01_EDA.ipynb           # Exploratory data analysis
    ├── 02_preprocessing.ipynb  # Feature matrix walkthrough
    ├── 03_kmeans_random forest_xizhi.ipynb   # K-means: elbow analysis, cluster profiles; Random forest: feature importance
    └── 04_knn_simon.ipynb      # KNN models + full 3-way comparison
```

2. Data Description

This analysis will use the Song Recommendations Dataset, sourced from student-provided survey responses. The song dataset provides information on individual song preferences and the demographic backgrounds of the respondents. It includes 172 entries and contains 10 variables that capture various aspects of each recommendation.

The dataset features Timestamp (string): The date and time the recommendation was submitted. Your Unique ID (string): A unique, consistent identifier for each respondent. Your Gender (string): The gender identity of the user. Your hometown (string): The type of environment where the user was raised (e.g., City, Suburban, Rural). What language do you primarily use in daily life? (string): The primary

language or languages spoken by the respondent. Song name (string): The title of the recommended musical track. Artist (string): The musician or group who performed the song. Genre (string): The musical category or style of the song. Language of the song (string): The language(s) featured in the song's lyrics. Song release year (string): The time period or specific era of the song's release (e.g., 2020+, 2010–2019).

Dataset

Song Recommendations.csv - 172 responses, 120 unique users, 170 unique songs.

Column	Description
User ID	Anonymous student identifier
Gender	Male / Female
Hometown	City / Suburban / Rural
User Language	Primary daily language(s)
Song / Artist	Submitted song
Genre	14 genre categories (after cleaning)
Song Language	Language(s) of the song
Release Year	Era bin: Pre-1960 -> 2020+

3. Data Preprocessing

Data Preprocessing

Cleaning Methodology The dataset was filtered to a final count of 170 valid records. Two samples were dropped because they contained missing or incomplete entries in essential fields such as the song name or primary genre, which are required for similarity calculations.

Standardizing text strings consolidated diverse user inputs into 14 distinct genres and 9 song language categories.

Variable names were systematically cleaned using prefixes such as genre_, slang_, and ulang_ to distinguish between song and user attributes and prevent naming conflicts during encoding.

A final integrity check verified that the feature matrix contained zero missing (NaN) values.

Feature Engineering

Multi-hot and one-hot encoding were used for nominal categorical variables to allow for multiple attributes per entry, such as a song belonging to more than one genre.

Multi-hot encoding represents data by assigning 1.0 to each applicable category and 0.0 to others.

Ordinal encoding was applied to temporal data to preserve the chronological relationship between release eras.

The release_year_ord column mapped specific intervals to sequential values: 3.0 for "2000-2009", 4.0 for "2010-2019", and 5.0 for "2020+".

User demographics and song features were concatenated into a 34-dimensional joint numerical feature matrix.

Model Justification Distance-based algorithms like KNN and K-Means require numerical inputs to calculate spatial proximity using metrics like cosine similarity or Euclidean distance.

Multi-hot encoding is necessary for these models to process categorical data without assuming a false mathematical hierarchy between independent groups like "Pop" and "Rock".

Ordinal encoding ensures that distance-based models recognize the relative temporal proximity between release years.

Tree-based models like Random Forest require numerical features to perform threshold splits during training.

Using ordinal values for years allows Random Forest to make logical chronological splits (e.g., year >= 4.0) to capture modern listening trends.

4. EDA: Explaoratory Data Analysis

- Data Issues

The raw dataset contained incomplete records, necessitating the removal of two samples with missing entries in critical fields, such as the song name or genre, to ensure accurate similarity calculations. Additionally, inconsistent text inputs required standardization to consolidate the data into clean categories.

- Patterns & Characteristics

Visualizations of the dataset revealed several distinct trends. The genre distribution demonstrated that Pop is the most prevalent genre, comprising approximately 37% of the dataset, followed by Rock and R&B/Soul. Demographic correlations indicated that Suburban users slightly favor Pop music, while City users exhibit more diverse genre preferences. Furthermore, the dataset exhibits a strong recency bias, with approximately 47% of the submitted songs released in the 2020+ era. There is also a massive user language skew toward English, spoken by roughly 99% of the users, with Chinese and Hindi appearing as the next most common languages. These distinct demographic and temporal patterns justify the inclusion of user attributes alongside song features to effectively cluster and recommend music.

- Data Preprocessing

Cleaning Methodology During the data cleaning phase, the dataset was filtered to a final count of 170 valid records. Two samples were dropped because they contained missing or incomplete entries in essential fields such as the song name or primary genre, which are required for subsequent similarity calculations. Standardizing the text strings successfully consolidated diverse user inputs into 14 distinct genres and 9 song language categories. To prevent naming conflicts during encoding and clearly distinguish between song and user attributes, variable names were systematically cleaned using prefixes such as genre_, slang_, and ulang_. Finally, an integrity check verified that the resulting feature matrix contained zero missing (NaN) values.

- Feature Engineering

To prepare the data for modeling, multi-hot and one-hot encoding were utilized for nominal categorical variables. This approach allowed for multiple attributes per entry, such as representing a single song that belongs to multiple genres, by assigning a value of 1.0 to each applicable category and 0.0 to others. For temporal data, ordinal encoding was applied to preserve the chronological relationship between release eras. Specifically, the release_year_ord column mapped intervals to sequential values, assigning 3.0 for “2000-2009”, 4.0 for “2010-2019”, and 5.0 for “2020+”. Ultimately, user demographics and song features were concatenated into a comprehensive 34-dimensional joint numerical feature matrix.

- Model Justification

These preprocessing steps were specifically tailored to support our chosen machine learning models. Distance-based algorithms like KNN and K-Means require numerical inputs to calculate spatial proximity using metrics such as cosine similarity or Euclidean distance. Multi-hot encoding is essential for these models to process categorical data without falsely assuming a mathematical hierarchy between independent groups, like “Pop” and “Rock”. Furthermore, ordinal encoding ensures that distance-based models correctly interpret the relative temporal proximity between release years. For tree-based models like Random Forest, which require numerical features to perform threshold splits during training, utilizing ordinal values for years allows the algorithm to make logical chronological splits (e.g., year >= 4.0) and effectively capture modern listening trends.

Approaches Simon - KNN Recommendation Content-based filtering (content_knn.py)

Each song is one-hot encoded (genre, language, release era).

A user’s profile = mean vector of their submitted songs.

Recommendations ranked by cosine similarity to the profile.

User-based collaborative filtering (collab_knn.py)

Each user is encoded by demographics (gender, hometown, language).

K nearest demographic neighbours are found via cosine similarity.

Songs submitted by neighbours (not yet seen by the user) are recommended.

Xizhi - K-means Clustering & Random Forest Cluster-based recommendation (kmeans_recommender.py)

Each row is represented as a joint vector: user demographics + song features.

Features scaled with StandardScaler; optimal K chosen by silhouette score.

A new user is assigned to the nearest cluster centroid, and the most popular songs in that cluster are recommended.

Limitation: K-means clustering is less efficient and effective for this dataset due to the high-dimensional, sparse nature of the multi-hot encoded categorical data (the “curse of dimensionality”). This sparsity dilutes the meaning of distance metrics and forces rigid, unpersonalized cluster assignments.

Supervised classification (random_forest_recommender.py)

Frames recommendation as a classification problem, predicting the likelihood of a user engaging with a particular song based on the joint feature matrix.

Handles the sparse multi-hot columns and ordinal temporal data natively by making logical, discrete threshold splits (e.g., isolating the presence of a specific genre or filtering by release_year_ord >= 4.0).

Recommendations are generated by scoring unseen songs and ranking them based on the tree ensemble’s predicted probability of a positive match.

5. User Preference Analysis

Methodology Cross-tabulation with row-wise normalization was utilized and visualized as Seaborn heatmaps to account for the uneven distribution of user demographics and reveal true proportional preferences.

Hometown vs. Genre Preferences

Suburban Users: Exhibit a strong preference for Pop music, representing 0.41 of their normalized distribution. This is higher than both Rural (0.38) and City (0.33) users.

City Users: Display a broader, more diverse listening profile. While Pop remains the top genre at 0.33, their secondary preferences are evenly split between Hip Hop/Rap (0.13) and Indie/Alternative (0.13), followed closely by R&B/Soul (0.10) and Rock (0.10).

Gender vs. Genre Preferences

Female Users: Show a highly concentrated preference for Pop music at 0.49, with a sharp drop-off to their second most popular genre, Indie/Alternative, at 0.12.

Male Users: Demonstrate a more varied genre distribution. Their preference for Pop is significantly lower at 0.27, followed closely by Rock (0.21) and Hip Hop/Rap (0.15).

Language Correlation

While English dominates the user base (~99%), minority language speakers also form distinct demographic segments. Approximately 35 users speak Chinese, and roughly 10 speak Hindi (similar to the proportion of Spanish speakers).

Summary Insight These specific correlations directly validate the modeling strategy. The distinct genre distributions across genders, hometowns, and languages justify utilizing user attributes as joint features in the Collaborative Filtering and K-Means models to successfully recommend songs even when prior listening history is sparse.

6. Recommendation Method

6.1 K-Nearest Neighbors (KNN) ApproachesSimilarity Measure: Both KNN models strictly utilize cosine similarity to compute distances.Content-Based Filtering: The system generates a user profile vector by calculating a simple mathematical average of the one-hot encoded feature vectors (genre, language, release era) of all songs submitted by the user. No temporal or preference weighting is applied. Recommendations are generated by calculating the cosine similarity between this mean user vector and the vectors of unseen songs in the dataset.User-Based Collaborative Filtering: The system calculates the cosine similarity between user demographic vectors (gender, hometown, language). It identifies the K=5 most similar demographic neighbors and recommends songs submitted by these neighbors that the target user has not yet seen.Output & Tuning: Both models output a default of Top K=5 song recommendations. Neighbor count variations (K in {3, 5, 10, 15, 20}) were evaluated to analyze the effect on genre match accuracy.

6.2 Random Forest Classification

Methodology & Output Logic: This approach frames recommendation as a supervised binary classification task predicting the likelihood of user engagement (Match vs. Non-match). To generate recommendations, the system evaluates the entire pool of unseen songs against the user's joint feature matrix. The tree ensemble assigns a predicted probability score for a positive match to each candidate song. The system then sorts the pool in descending order based on these probability scores, outputting the highest-ranked songs as the final recommendations.

Hyperparameters: Optimized via GridSearchCV (48 combinations). The optimal settings are n_estimators = 300, max_depth = 20, and min_samples_split = 2.

Feature Importance: Gini importance evaluation identified hometown_Suburban as the strongest demographic predictor, gender_Female as highly predictive among user features, and genre_Pop as the dominant song-content feature.

6.3 K-Means Clustering Methodology: The system utilizes sklearn.cluster.KMeans to assign users to clusters. The 34-dimensional joint feature matrix (user demographics and song preferences) is first pre-processed using StandardScaler (Z-score normalization) to ensure equitable feature influence. Centroid assignment is then calculated utilizing Euclidean distance.Recommendation Logic: The system employs a cluster-based popularity ranking algorithm. A new user is mapped to the nearest of the K=10 cluster centroids. The system filters the dataset to identify all songs submitted by other users within that specific cluster and excludes any songs the target user has already seen. The remaining songs are aggregated by frequency count. The system outputs the top 5 most popular songs within that cluster, utilizing dataframe appearance order for tie-breaking.Evaluation & Limitations: The model achieved a low Genre Match Accuracy of 0.1529. This underperformance is directly attributable to the highly unbalanced cluster distribution observed during evaluation (e.g., Cluster 1 contains 59 users, while Cluster 3 contains 1). In heavily disproportionate clusters, the "popularity within cluster" logic fails: massive clusters regress to recommending generic, dataset-wide popular songs rather than personalized matches, while undersized clusters lack sufficient data to generate reliable frequency rankings.

7. Handling Out-of-Vocabulary (OOV) Songs & Recommending

Interactive Metadata Prompting and Feature Imputation When a user queries a song that does not exist in the dataset (an Out-of-Vocabulary or OOV song), the system cannot natively compute recommendations because it lacks the mathematical feature vector for that specific track. To resolve this, the system implements an interactive imputation strategy to generate a representative proxy vector:

Detection: The system checks the inputted song string against the dataset index. If missing, the OOV protocol is triggered.

Interactive Prompting: The user is notified of the missing song, presented with a list of valid dataset genres, and prompted to input the genre of their requested track.

Primary Imputation (Genre Mean): If a valid genre is provided, the system isolates all existing songs belonging to that genre and calculates the mathematical mean of their feature vectors. This proxy vector represents the typical profile of a song within that specific category.

Fallback Imputation (Global Mean): If the user inputs an invalid or unrecognized genre, the system calculates a global average vector across the entire dataset to ensure the recommendation pipeline does not fail.

Analysis for the K-Nearest Neighbors (KNN) Approach In the KNN pipeline, the imputed proxy vector serves as the direct numerical representation of the unknown song. The system computes the cosine similarity between this newly generated proxy vector and the feature vectors of all existing songs in the dataset. By calculating the angular distance, the system identifies and extracts the top K nearest neighbors. This mechanism guarantees that even without historical data for the specific track, the resulting recommendations mathematically align with the average characteristics and feature weights of the user-designated genre.

Analysis for the Random Forest Approach The Random Forest model relies on a complete joint feature matrix to execute its supervised classification tasks. The imputation protocol transforms the OOV song into a standardized numerical format that the pre-trained decision trees can process natively. The model evaluates this proxy vector against the user's demographic features, passing the imputed data through its ensemble of threshold splits (e.g., evaluating the presence of the specified genre). Instead of relying strictly on spatial distance, the ensemble generates a predicted probability score for user engagement, allowing the system to rank and recommend relevant songs based on predictive matching logic.

K-Means Clustering Exclusion OOV handling was intentionally excluded from the K-Means Clustering pipeline. The evaluation phase demonstrated that the K-Means model possessed weak structural integrity (a silhouette score of 0.2023) and produced highly disproportionate cluster assignments. Forcing an imputed proxy vector into these poorly defined, unbalanced clusters would yield arbitrary and unreliable recommendations, rendering the OOV strategy ineffective for this specific model.

Sample Result for KNN in --- Testing OOV Song Recommendations --- [-] Song 'Fake Plastic Trees (Radiohead)' not found in the dataset. Available genres: Pop, Indie / Alternative, Electronic / Dance, Hip Hop / Rap, Rock, Alternative Rock, K-pop, R&B / Soul, Mariachi, Classical, Country, Latin, Video Game Music, Jazz Enter the genre of the new song: Rock [+] Imputing features using the average of existing 'Rock' songs.

Top 5 Recommendations based on 'Fake Plastic Trees (Radiohead)': song similarity artist genre release_year 0 Friday I'm in Love 0.996052 The Cure Rock 1980-1999 1 Hold My Hand 0.996052 Hootie & The Blowfish Rock 1980-1999 2 No More Tears 0.996052 Ozzy Osbourne Rock 1980-1999 3 Paranoid Android 0.996052 Radiohead Rock 1980-1999 4 Island in the Sun 0.994373 Weezer Rock 2000-2009

Sample Reuslt for Random forest

--- Testing OOV Song Recommendations --- [-] Song 'Fake Plastic Trees (Radiohead)' not found in the dataset. Available genres: Pop, Indie / Alternative, Electronic / Dance, Hip Hop / Rap, Rock, Alternative Rock, K-pop, R&B / Soul, Mariachi, Classical, Country, Latin, Video Game Music, Jazz Enter the genre of the new song: Rock [+] Imputing features using the average of existing 'Rock' songs.

Top 5 Recommendations based on 'Fake Plastic Trees (Radiohead)': song similarity artist genre 0 Paranoid Android 0.996193 Radiohead Rock 1 Hold My Hand 0.996193 Hootie & The Blowfish Rock 2 No More Tears 0.996193 Ozzy Osbourne Rock 3 Friday I'm in Love 0.996193 The Cure Rock 4 Island in the Sun 0.994514 Weezer Rock

Since the K-Means clustering is not so good in previous performance, we did not do the OOV situation for K-Means Clustering.

8. Example Results

Sample Result for KNN - Content-Based User: mariokart Submitted songs: song genre release_year TITFORTAT Pop 2020+ Birds of a Feather Pop 2020+ The Subway Pop 2020+ BAILE INoLVIDABLE

Latin 2020+ Rolling in the Deep R&B / Soul 2010-2019 Getaway Car Rock 2020+ Sign of the Times Rock 2010-2019 Hurricane Pop 2010-2019 Takedown K-pop 2020+ Meant to be Country 2010-2019

Content-based top-5 recommendations: song artist genre release_year similarity Fame is a gun Addison Rae Pop 2020+ 0.99187 run for the hills Tate McRae Pop 2020+ 0.99187 Murder on the dancefloor Sophie Ellis-bextor Pop 2020+ 0.99187 Levitating Dua Lipa Pop 2020+ 0.99187 Pink pony club Chappell Roan Pop 2020+ 0.99187

Content-Based Filtering recommends items by computing similarity between the features of potential items and the user's aggregated feature profile. The user's submitted history is heavily skewed toward the "Pop" genre and the "2020+" release era. The algorithm captured this dominant cluster, outputting entirely 2020+ Pop songs to reflect a direct matching of the most frequent categorical variables. The identical high similarity scores of 0.99187 indicate these recommended tracks share identical feature representations. These representations align closely with the user's mean feature vector, limiting the diversity of the final output.

Sample Result for KNN - Collaborative User: mariokart Collaborative top-5 recommendations: song artist genre neighbor_count Let Alone the One You Love Olivia Dean Pop 1 Pink pony club Chappell Roan Pop 1 Reality TV Argument Bleeds Wednesday Indie / Alternative 1 So Easy To Fall In Love Olivia Dean Pop 1 Tv Off Kendrick Lamar Hip Hop / Rap 1

Collaborative Filtering recommends items by identifying similar users based on overlapping interaction history and suggesting items those specific neighbors liked. The generated recommendations exhibit greater genre diversity, including Pop, Indie / Alternative, and Hip Hop / Rap. The user's input history contains eclectic secondary preferences, allowing the algorithm to isolate a nearest neighbor sharing overlap with this diverse background. The output consists of distinct songs this specific neighbor preferred, bypassing strict item metadata constraints. This process crosses established genre boundaries to deliver recommendations based entirely on user-to-user behavioral correlations.

Sample Result for K-Means Clustering

User: mariokart Known songs: song genre TITFORTAT Pop Birds of a Feather Pop The Subway Pop BAILE INoLVIDABLE Latin Rolling in the Deep R&B / Soul Getaway Car Rock Sign of the Times Rock Hurricane Pop Takedown K-pop Meant to be Country

K-means recommendations: song artist genre count 0 Paris Chainsmokers Pop 1 1 Reality TV Argument Bleeds Wednesday Indie / Alternative 1

The K-means algorithm groups user 'mariokart' into a specific cluster based on their joint demographic and listening profile, heavily anchored by mainstream pop tracks. Recommending "Paris" by The Chainsmokers aligns directly with this dominant pop preference, reflecting the cluster's aggregate genre orientation. Suggesting the Indie/Alternative track "Reality TV Argument Bleeds" highlights a distinct limitation of this unsupervised clustering method. Because K-means relies on broad cluster popularity rather than strict individual history, it risks surfacing tracks popular among demographically similar users regardless of personal genre mismatch. The algorithm outputs a genre entirely absent from the user's known history, demonstrating how centroid-based recommendations can dilute personalization and produce nonsensical outputs.

Sample Result for Random Forest

Recommendations for user: mariokart song probability artist genre 0 They call this love 1.0 Mathew Ifield Pop 1 End of Beginning 1.0 Djo Pop 2 West End Girl 1.0 Lily Allen Pop 3 Love Me Not 1.0 Ravyn Lenae Pop 4 Open Arms 1.0 SZA Pop

Random Forest model predicts user-item interactions by analyzing nonlinear decision boundaries between specific user traits and song features. Recognizing that 40% of the user's known history belongs to the Pop genre, the algorithm identifies this attribute as the primary driver for future positive interactions. The model subsequently outputs a homogenous list of Pop tracks, assigning absolute certainty (probability 1.0) to artists like SZA and Lily Allen. This strict categorization makes sense because the supervised learning framework isolated the strongest predictive features to minimize classification error for 'mariokart' individually. The model avoids the noisy generalization of K-means, resulting in a highly targeted, historically consistent recommendation list devoid of out-of-genre anomalies.

Conclusion: Overall Model Performance When evaluating the Top-5 recommendation performance across all models, Content-Based KNN emerged as the clearly superior approach, achieving an exceptional primary genre match rate of 0.9545. User-Based Collaborative KNN (0.4545) and the Random Forest classifier (0.4224) demonstrated comparable mid-tier performance in maintaining genre relevance, while K-Means Clustering proved to be the least effective system, yielding a genre accuracy of only 0.1529 due to unbalanced cluster assignments. Notably, while the dataset's extreme sparsity (170 unique songs across 172 rows) drove the exact hit rate for both KNN models to 0.0000-as held-out songs rarely existed in other users' pools-the Random Forest model was the only system resilient enough to overcome this sparsity and achieve a positive hit rate (0.1379). Ultimately, if strict categorical similarity is the goal, Content-Based KNN is the optimal choice, whereas Random Forest offers a more robust framework for predicting exact item engagement in sparse environments.

Setup

```
pip install pandas numpy scikit-learn matplotlib seaborn jupyter
```

Run notebooks in order (01 -> 04) from the project root, or import from src/ directly:

```
import sys; sys.path.insert(0, 'src')
from preprocess import load_and_clean, build_song_features, build_user_features
```